

SmartLock

Présentation du projet :

- *Le projet SmartLock modernise la gestion du matériel du DeVinci Fablab en remplaçant les serrures traditionnelles par un système d'accès sécurisé et connecté via badge NFC.*
- *Cette solution couple la sécurisation physique des armoires à une gestion centralisée des stocks pour garantir une traçabilité complète des emprunts en temps réel.*
- *A terme, la solution sera implémentée toutes les armoires du Fablab afin de sécuriser le matériel de l'espace.*

V4 - VF	Design Détaillé — Architecture Informatique	14/12/2025
Rédacteur : Florian Cazac Yann Vidamment	Relecteurs : Augustin Wintrebert, Guilhem Henoux, Alexandre Houdin	Jalon de design détaillé
	Validateurs : Florian Cazac	





Table des matières

1.	Introduction	3
2.	Architecture globale.....	3
2.1.	Infrastructure Serveur	3
2.1.1.	Authentification.....	3
2.1.2.	Base de Données.....	4
2.1.3.	Frontend d'Administration	5
2.1.4.	Backend API.....	5
2.1.5.	Déploiement et Conteneurisation.....	5
2.2.	Interface Client.....	6
2.2.1.	Interface Homme-Machine (IHM)	6
2.2.2.	Contrôleur Matériel Hardware Abstraction Layer	6
3.	Architecture Serveur	7
3.1.	Besoins fonctionnels minimaux du serveur.....	7
3.1.1.	Spécifications matérielles	7
3.1.2.	Environnement logiciel et sécurité.....	7
3.2.	Schéma fonctionnel des services sur le serveur.....	7
3.2.1.	Architecture des services	7
3.2.2.	Flux de communication	8
4.	Architecture Microprocesseur	9
4.1.	Environnement.....	9
4.2.	Détail des Modules.....	9
4.2.1.	Module HAL (Hardware Abstraction Layer).....	9
4.2.2.	Module UI	9
4.2.3.	Module Session & Logique.....	9
4.2.4.	Module API Client	9
4.3.	Communication	10
4.3.1.	Communication Matérielle.....	10
4.3.2.	Communication Externe.....	10
4.4.	Installation et Configuration.....	10
4.5.	Gestion des Erreurs et Résilience	10



5. UI/UX de l'interface client.....	11
5.1. Parcours utilisateur et flux d'interaction.....	11
5.2. Écran d'accueil.....	11
5.3. Écran de navigation et sélection.....	12
5.4. Écran de gestion d'un produit.....	13
5.5. Écran de validation finale.....	13
5.6. Écran d'accès physique à l'armoire.....	14
5.7. Écran de clôture de transaction.....	14
6. Interface API.....	15
6.1. CRUD Item.....	15
6.2. CRUD Category.....	15
6.3. CRUD Locker.....	16
6.4. Stock & Mouvements.....	16
6.4.1. Consulter l'inventaire.....	16
6.4.3. Ajuster le stock (Admin).....	17
6.5. CRUD Stock.....	18
6.5.1. Consulter le stock.....	18
6.5.2. Ajuster le stock (Admin).....	19
6.6. Authentification + Autorisation (NFC).....	19
6.6.1. Accès à l'armoire via NFC.....	19
6.6.2. Armoires accessibles par l'utilisateur.....	21
6.7. Gestion des Permissions (Admin).....	22
6.8. Audit & Logs.....	23
6.9. Code HTTP.....	24



1. Introduction

Ce document définit l'architecture informatique globale, serveur et logicielle détaillée du client embarqué pour le projet SmartLock.

2. Architecture globale

L'architecture logicielle du projet est conçue pour assurer une séparation claire entre le serveur central et le client embarqué sur l'armoire, garantissant ainsi modularité et facilité de maintenance.

2.1. Infrastructure Serveur

Le serveur central est le cerveau du système, hébergé sur une machine distante.

2.1.1. Authentification

Architecture d'authentification

Le système d'authentification du casier intelligent repose sur un compte de service (authentification machine-à-machine) qui interroge l'API Admin de Keycloak

Séquence d'authentification

Lorsqu'une carte NFC est scannée, le processus suivant est exécuté :

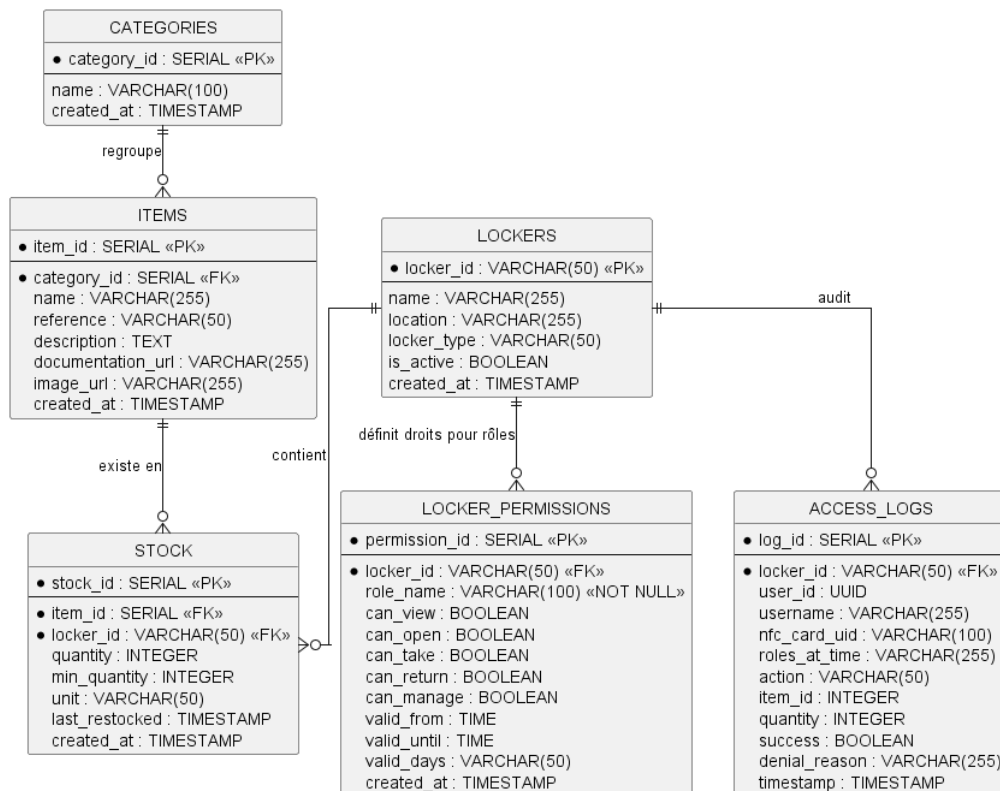
1. **Lecture** : Capture de l'UID de la carte NFC
2. **Requête** : Envoi de l'UID à l'API backend du casier
3. **Authentification** : Le système s'authentifie auprès de Keycloak avec les identifiants du compte de service
4. **Recherche** : Identification de l'utilisateur possédant l'attribut *nfc_card_id* correspondant
5. **Récupération** : Obtention des rôles et permissions associés à l'utilisateur
6. **Autorisation** : Décision d'accès au casier basée sur les rôles de l'utilisateur
7. **Traçabilité** : Enregistrement de la transaction avec le contexte utilisateur complet

Nous avons fait le choix pragmatique d'utiliser Keycloak dans sa version standard, en exploitant simplement son API Admin et ses comptes de service natifs. Cette approche nous évite tout développement spécifique complexe : le système est ainsi plus robuste, facile à maintenir, et les mises à jour se font sans risque de régression. Il centralise la gestion des identités, qu'il s'agisse de l'accès web (SSO) ou des badges NFC, assurant une cohérence totale des droits.



2.1.2. Base de Données

PostgreSQL a été retenu comme système de gestion de base de données relationnelle pour assurer la persistance des données métier. Cette solution stocke de manière structurée les inventaires, les configurations des casiers et l'historique complet des accès.



CATEGORIES : Nomenclature simple permettant d'organiser les produits par familles (ex : "Consommables") pour faciliter la recherche et le filtrage dans l'application.

ITEMS : Le catalogue de référence des produits. Elle décrit "ce que c'est" de manière générique (nom, photo, manuel PDF, référence constructeur), indépendamment de l'endroit où l'objet est stocké ou de la quantité disponible.

LOCKERS : Représente les armoires physiques intelligentes installées. Elle identifie chaque machine de manière unique via son ID technique et précise son emplacement géographique et son type.

STOCK : L'état réel de l'inventaire. C'est la table de liaison qui indique "combien" d'unités d'un produit (Item) sont présentes dans une armoire spécifique (Locker), et gère les seuils d'alerte pour le réapprovisionnement.



LOCKER_PERMISSIONS : Le moteur de contrôle d'accès (ACL). Elle définit les règles de sécurité en précisant qui (un utilisateur précis ou un rôle entier comme "équipe de nuit") a le droit d'effectuer quelles actions (voir, ouvrir, prendre) sur quel casier, éventuellement sur des plages horaires définies.

ACCESS_LOGS : Le journal d'audit immuable du système. Elle trace chronologiquement toutes les tentatives d'interaction (réussies ou refusées), en enregistrant l'utilisateur, la carte utilisée, l'action tentée et le casier concerné pour des besoins de sécurité et de traçabilité.

2.1.3. Frontend d'Administration

L'interface web d'administration est développée avec **Svelte**, un framework **JavaScript** moderne. Cette interface permet la visualisation en temps réel de l'état des stocks et des armoires. Les administrateurs peuvent gérer de manière centralisée les configurations des armoires, les droits d'accès et les associations entre badges NFC et comptes utilisateurs.

2.1.4. Backend API

Le backend est développé en **Python** avec le framework **FastAPI**. Ce choix technologique permet d'exposer une API RESTful performante et asynchrone, capable de traiter les requêtes d'autorisation d'accès en temps réel. Le framework génère automatiquement une **documentation**, facilitant l'intégration avec les clients et les tests de l'API.

2.1.5. Déploiement et Conteneurisation

L'ensemble de la stack serveur (Base de données, API, Frontend, Keycloak) est conteneurisé à l'aide de Docker et orchestré via Docker Compose.



2.2. Interface Client

Chaque armoire intelligente est équipée d'un Raspberry Pi Zero 2 W assurant l'autonomie fonctionnelle du système en local. Ce nano-ordinateur constitue l'interface entre le matériel (lecteurs NFC, Verrou, écran) et le serveur central.

2.2.1. Interface Homme-Machine (IHM)

Une application graphique développée avec **CustomTkinter** offre une expérience utilisateur moderne et intuitive. Cette surcouche du framework TkInter standard permet de créer des interfaces visuellement attractives, adaptées à un écran tactile de 7 pouces. L'application affiche des retours visuels clairs concernant le statut d'autorisation (accès accordé ou refusé) ainsi que les indicateurs de connectivité réseau.

2.2.2. Contrôleur Matériel Hardware Abstraction Layer

En arrière-plan de l'interface graphique, un service Python dédié gère les interactions physiques via les ports GPIO du Raspberry Pi. Ce module assure deux fonctions critiques :

- **Lecture NFC** : Interfaçage avec le lecteur de badge via le protocole I2C pour récupérer l'UID des cartes en temps réel.
- **Pilotage des verrous** : Le Raspberry Pi ne pouvant alimenter directement les serrures, il pilote un module de MOSFET. Ce circuit de puissance agit comme un interrupteur commandé, permettant d'envoyer l'alimentation 12V nécessaire aux solénoïdes pour déverrouiller physiquement les casiers.



3. Architecture Serveur

3.1. Besoins fonctionnels minimaux du serveur

3.1.1. Spécifications matérielles

L'infrastructure requiert les ressources minimales suivantes :

- Mémoire RAM : 4 Go minimum
- Processeur : 2 cœurs à 3 GHz
- Stockage : 10 Go + 20 Go de marge d'exploitation pour plusieurs années d'utilisation

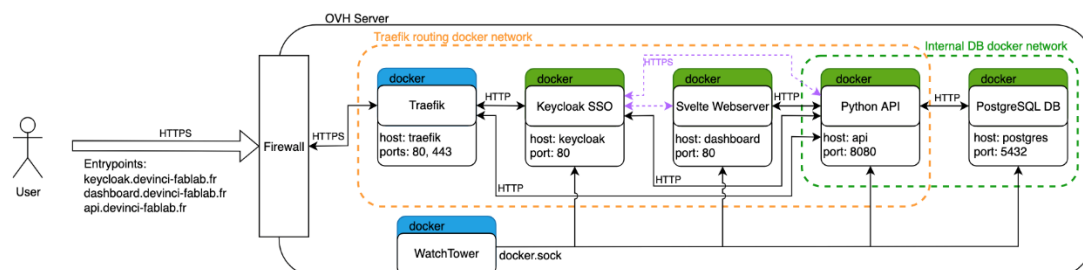
État actuel : Le serveur du FabLab couvre intégralement ces besoins. Aucune mise à niveau matérielle n'est nécessaire.

3.1.2. Environnement logiciel et sécurité

L'infrastructure s'appuie sur les composants de sécurité suivants :

- Conteneurisation Docker
- Pare-feu système (firewall)
- Chiffrement SSL/TLS
- Sauvegardes régulières automatisées

3.2. Schéma fonctionnel des services sur le serveur



3.2.1. Architecture des services

L'infrastructure repose sur une architecture microservices conteneurisée comprenant :

Services existants (représentés en bleu) :

- Traefik (reverse proxy et routage)
- Watchtower (mise à jour automatique)

Services ajoutés par Smartlock (représentés en vert) :



- Keycloak (gestion d'identité)
- Serveur Web (interface utilisateur)
- API backend
- Base de données (PostgreSQL)

3.2.2. Flux de communication

Accès externe : Les utilisateurs accèdent aux services (Serveur Web, API, Keycloak) via Traefik, qui route les requêtes à travers le pare-feu système (Iptables).

Communications internes :

- Le serveur Web interroge l'API pour toutes les opérations métier
- L'API est le seul point d'accès à la base de données, assurant l'isolation des données
- Keycloak gère l'authentification via SSO pour les utilisateurs Web et via un compte de service pour l'authentification par badge NFC depuis l'API

Watchtower surveille et met à jour automatiquement les services dès la publication de Nouvelles images Docker (accès en lecture/écriture au socket Docker) pour les conteneurs Smartlock uniquement.

Protocoles de communication :

Réseau interne : HTTP entre les conteneurs

Réseau externe : HTTPS uniquement, sécurisé par Traefik avec certificats SSL/TLS

Cas particulier - Sécurité renforcée de Keycloak : Pour des raisons de sécurité accrues, Keycloak n'est accessible aux autres services qu'en HTTPS. Les services internes accèdent à Keycloak via son URL publique (après routage par Traefik) plutôt que par communication interne directe. Ce flux de trafic spécifique est représenté par des pointillés violets sur le schema.



4. Architecture Microprocesseur

4.1. Environnement

- Matériel : Raspberry Pi Zero 2 W.
- OS (Système d'Exploitation) : Raspberry Pi OS Lite.
- Langage : Python 3.11+ pour garder une cohérence de langage (API et Frontend Client)
- Gestionnaire de Service : Systemd.

4.2. Détail des Modules

4.2.1. Module HAL (Hardware Abstraction Layer)

Bibliothèques : gpiozero (Mosfet/LED), adafruit-pn532 (NFC).

Le module HAL isole les interactions physiques du reste du programme. Il intègre une classe *NFCReader* qui opère dans un thread indépendant pour garantir la fluidité de l'interface, communiquant via le protocole SPI. Parallèlement, la classe *LockerController* gère les broches GPIO pour piloter le verrouillage et lire les capteurs sans bloquer l'exécution principale.

4.2.2. Module UI

Bibliothèque : CustomTkinter (CTk).

Le module UI, basé sur *CustomTkinter*, utilise un contrôleur central pour gérer la navigation entre les différents écrans (Accueil, Sélection, Panier, Feedback). Chaque vue est instanciée comme un cadre interchangeable, permettant une transition fluide adaptée au parcours utilisateur défini dans les wireframes.

4.2.3. Module Session & Logique

La classe *SessionManager* conserve l'état de l'utilisateur connecté et le contenu de son panier. Elle implémente une sécurité active via un minuteur d'inactivité : l'utilisateur est déconnecté après 90 secondes, et l'écran est mis en veille après 30 secondes supplémentaires pour économiser l'énergie.

4.2.4. Module API Client

Bibliothèque : requests

Rôle : Encapsule les appels REST vers le serveur. Gère les Timeouts (si le wifi coupe) et les erreurs HTTP (404, 500).



4.3. Communication

4.3.1. Communication Materielle

- NFC (PN532) : I2C
- Mosfet : GPIO (Sortie Numérique 3.3V).
- Écran Tactile : HDMI + USB (pour le tactile).

4.3.2. Communication Externe

- Protocole : HTTP/1.1 (REST API) avec payload JSON.
- Sécurité : HTTPS (TLS) fortement recommandé.
- Wifi : Utilisation de la puce Wifi intégrée au Zero 2 W.

4.4. Installation et Configuration

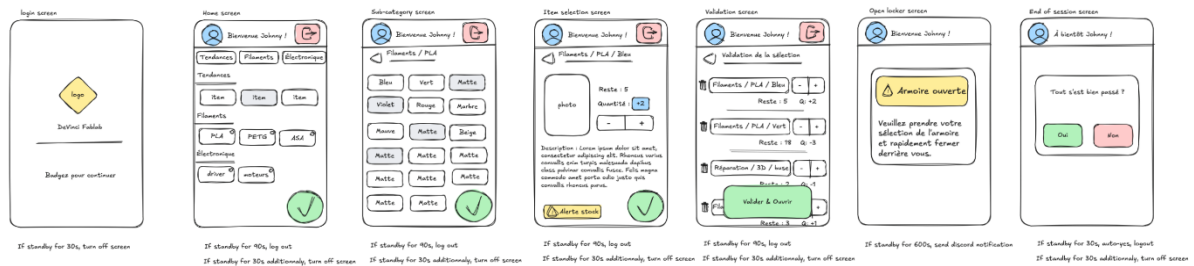
L'ensemble des dépendances est figé dans un fichier pyproject.toml, permettant une installation automatisée. Le fichier /boot/config.txt est ajusté pour forcer la résolution HDMI (1024x600) et activer l'interface I2C nécessaire au lecteur de badges.

4.5. Gestion des Erreurs et Résilience

En cas de coupure Wi-Fi, le module API Client intercepte les exceptions réseau. L'interface affiche alors une pop-up "Mode Hors Ligne" bloquant temporairement les nouvelles transactions pour éviter les désynchronisations de stock, tout en tentant une reconnexion automatique en arrière-plan.



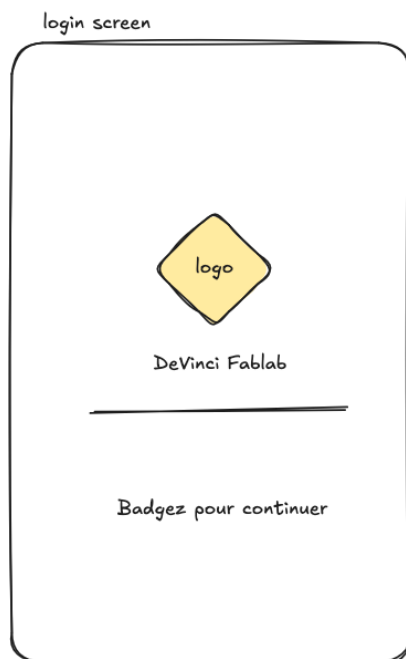
5. UI/UX de l'interface client



5.1. Parcours utilisateur et flux d'interaction

L'interface client du casier intelligent a été conçue selon une approche centrée utilisateur, privilégiant la simplicité et l'efficacité des interactions. Le parcours utilisateur se décompose en sept étapes distinctes, chacune optimisée pour garantir une expérience fluide et intuitive.

5.2. Écran d'accueil



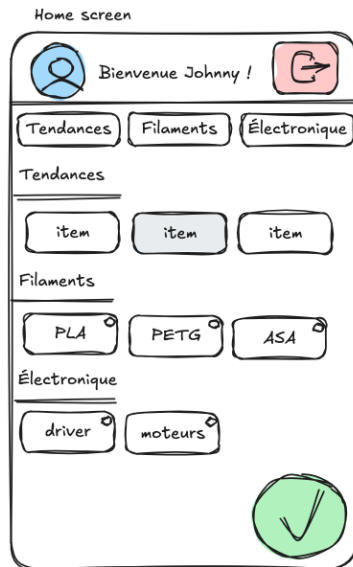
If standby for 30s, turn off screen

Description fonctionnelle

L'écran d'accueil constitue le point d'entrée du système. Il affiche une invitation claire à badger sa carte NFC pour démarrer une session. Par mesure de sécurité, aucune interaction n'est possible avant l'authentification de l'utilisateur, garantissant ainsi que seules les personnes autorisées peuvent accéder au système.



5.3. Écran de navigation et sélection

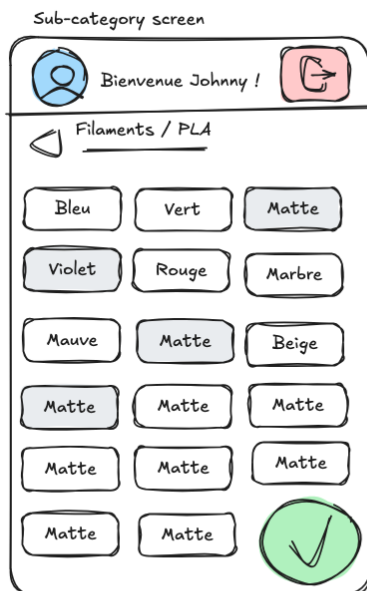


If standby for 90s, log out

If standby for 30s additionnaly, turn off screen

Vue des catégories de produits

Une fois l'authentification réussie, l'utilisateur accède à l'écran de sélection des catégories de produits, marquant le début de la navigation dans le catalogue. À ce stade, l'utilisateur peut parcourir les différentes catégories disponibles ou choisir de sortir de sa session. L'armoire reste verrouillée durant toute la phase de sélection, ne s'ouvrant qu'après validation finale des produits choisis.



If standby for 90s, log out

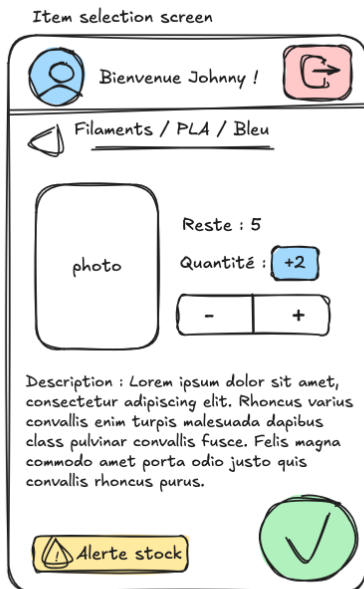
in If standby for 30s additionnaly, turn off screen

Vue galerie des articles

L'interface de sélection des articles adopte une présentation en galerie, offrant une vue claire et organisée des produits disponibles dans la catégorie sélectionnée. Cette disposition facilite la consultation rapide du catalogue et permet une identification visuelle immédiate des articles.



5.4. Écran de gestion d'un produit



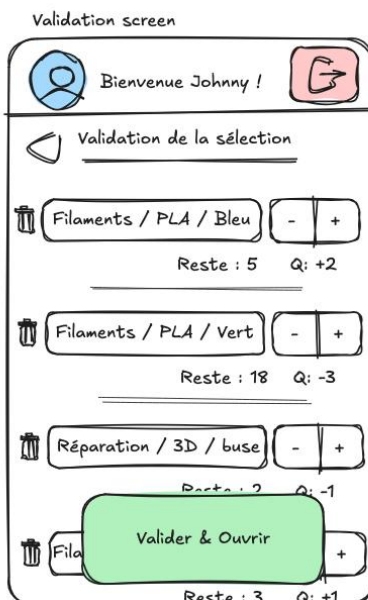
Fonctionnalités disponibles

Cet écran offre un ensemble complet de fonctionnalités pour la gestion des produits. L'utilisateur peut naviguer librement en arrière pour modifier sa sélection, ajuster les quantités à ajouter ou retirer du stock, et envoyer manuellement une alerte de stock bas. Ces alertes sont transmises directement au bureau du Trésorier pour un traitement rapide des besoins de réapprovisionnement.

If standby for 90s, log out

1 If standby for 30s additionnaly, turn off screen

5.5. Écran de validation finale



If standby for 90s, log out

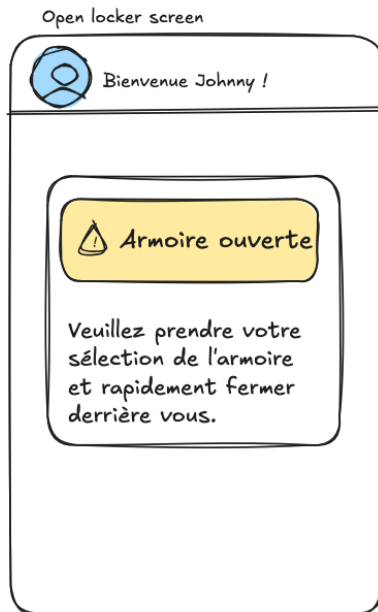
reen If standby for 30s additionnaly, turn off screen

Point de contrôle avant ouverture

L'écran de confirmation présente un récapitulatif complet de la sélection effectuée. Cette étape cruciale permet à l'utilisateur de vérifier l'ensemble de ses choix avant validation définitive. Un bouton de retour reste accessible pour permettre toute modification de dernière minute, garantissant ainsi la précision de la transaction.



5.6. Écran d'accès physique à l'armoire

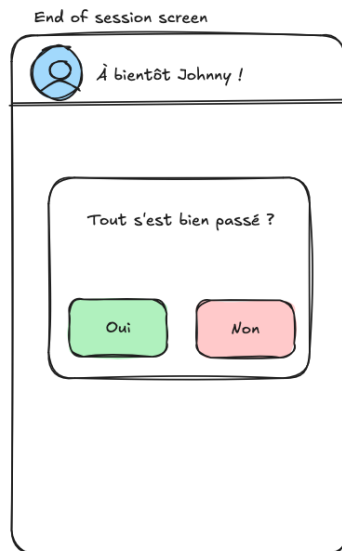


If standby for 600s, send discord notification

Phase de manipulation des produits

Suite à la validation de la sélection, l'armoire se déverrouille et l'utilisateur est invité à récupérer ou déposer les produits sélectionnés. Un système de surveillance d'inactivité est actif durant cette phase: si aucune action n'est détectée pendant plus de 10 minutes, une alerte automatique est envoyée sur le canal Discord dédié pour signaler une anomalie potentielle nécessitant une intervention.

5.7. Écran de clôture de transaction



on If standby for 30s, auto-yes, logout
If standby for 30s additionally, turn off screen

Validation et gestion d'incidents

L'écran final propose une dernière vérification de la transaction. Si l'utilisateur confirme que tout s'est déroulé correctement, la session se termine automatiquement et l'armoire se verrouille. En cas de problème (produit manquant, erreur de sélection), l'utilisateur dispose d'une option pour rouvrir temporairement l'armoire et effectuer les corrections nécessaires avant de finaliser définitivement la transaction.



6. Interface API

6.1. CRUD Item

- GET /items : Retourne tous les items
- GET /items/{item_id} : Retourne un item avec son id
- POST /items : Crée un item

```
{
  "name": "Perceuse",
  "category_id": 3,
  "reference": "PRC-22",
  "description": "Perceuse à percussion",
  "documentation_url": "https://example.com/docs/prc22.pdf",
  "image_url": "https://example.com/images/prc22.jpg"
}
```
- PUT /items/{item_id} : Met à jour un item
- DELETE /items/{item_id} : Supprime un item (vérifie qu'il n'est pas en stock)

6.2. CRUD Category

- GET /categories : Retourne toutes les catégories
- GET /categories/{category_id} : Retourne une catégorie avec son id
- POST /categories : Crée une catégorie

```
{
  "name": "Outillage électrique"
}
```
- PUT /categories/{category_id} : Met à jour une catégorie
- DELETE /categories/{category_id} : Supprime une catégorie (vérifie qu'elle n'a pas d'items)



6.3. CRUD Locker

- GET /lockers : Retourne toutes les armoires
- GET /lockers/{locker_id} : Retourne une armoire avec son id
- POST /lockers : Crée une armoire

```
{
  "name": "Armoire P - Partenariat",
  "location": "Bâtiment A",
  "locker_type": "warehouse"
}
```
- PUT /lockers/{locker_id} : Met à jour une armoire
- DELETE /lockers/{locker_id} : Supprime une armoire (soft delete : is_active = false)

6.4. Stock & Mouvements

6.4.1. Consulter l'inventaire

- GET /lockers/{locker_id}/inventory : Retourne les stocks d'une armoire

```
{
  "locker_id": "WH-001",
  "locker_name": "Armoire A - Atelier",
  "items": [
    {
      "stock_id": 10,
      "item_id": 1,
      "item_name": "Perceuse",
      "reference": "PRC-22",
      "category": "Outillage électrique",
      "quantity": 3,
      "min_quantity": 5
    }
  ]
}
```



6.4.3. Ajuster le stock

- POST /stock/in : Ajoute du stock (approvisionnement)

```
{  
  "item_id": 1,  
  "locker_id": "WH-001",  
  "quantity": 5  
}
```

Résultat :

- Met à jour STOCK.quantity += 5
- Crée log dans ACCESS_LOGS : action = "STOCK_IN", user_id = utilisateur connecté

- POST /stock/out : Retire du stock (prélèvement)

```
{  
  "item_id": 1,  
  "locker_id": "WH-001",  
  "quantity": 3  
}
```

Résultat :

- Met à jour STOCK.quantity -= 3
- Crée log dans ACCESS_LOGS : action = "STOCK_OUT", user_id = utilisateur connecté



6.5. CRUD Stock

6.5.1. Consulter le stock

- GET /stock : Liste tout le stock (avec filtres optionnels)
- GET /stock?locker_id=WH-001 // Stock d'une armoire
- GET /stock?item_id=1 // Partout où cet item existe
- GET /stock?min_quantity_alert=true // Items en alerte ($qty \leq min_qty$)
- GET /stock/{stock_id} : Détails d'une ligne de stock

```
{
  "stock_id": 10,
  "item_id": 1,
  "item_name": "Perceuse",
  "locker_id": "WH-001",
  "quantity": 3,
  "min_quantity": 5,
  "unit": "pcs",
  "last_restocked": "2025-01-10T14:30:00Z"
}
```

- GET /items/{item_id}/stock : Stock d'un item dans toutes les armoires

```
{
  "item_id": 1,
  "item_name": "Perceuse",
  "reference": "PRC-22",
  "stocks": [
    {
      "stock_id": 10,
      "locker_id": "WH-001",
      "locker_name": "Armoire A",
      "quantity": 3
    },
    {
      "stock_id": 15,
      "locker_id": "WH-002",
      "locker_name": "Armoire B",
      "quantity": 7
    }
  ]
}
```



6.5.2. Ajuster le stock (Admin)

- PUT /stock/{stock_id} : Ajuste directement la quantité

```
{  
  "quantity": 20,  
  "reason": "Inventory check correction"  
}
```

Résultat :

- Met à jour STOCK.quantity = 20
- Crée log dans ACCESS_LOGS : action = "STOCK_ADJUST", reason, user_id = utilisateur connecté

6.6. Authentification + Autorisation (NFC)

6.6.1. Accès à l'armoire via NFC

- POST /api/locker/access : Endpoint principal pour l'accès aux armoires

```
{  
  "nfc_uid": "04:AB:CD:EF",  
  "locker_id": "WH-001",  
  "action": "take",  
  "item_id": 10,  
  "quantity": 3  
}
```

Actions valides :

- "take" : Prélever des items (nécessite item_id et quantity)
- "return" : Retourner des items (nécessite item_id et quantity)
- "view" : Consulter l'armoire (optionnel : item_id)
- "manage" : Gérer l'armoire (admin only)



Flux :

1. L'API interroge Keycloak (via Compte de Service) pour trouver l'utilisateur possédant l'attribut *nfc_uid*
2. Keycloak retourne l'identité de l'utilisateur (UUID, Username) et la liste de ses rôles.
3. L'API interroge la table *LOCKER_PERMISSIONS*
4. Vérification Stock : Si action = "take", vérifie *STOCK.quantity*.
5. Effectue l'opération :
 - Met à jour *STOCK.quantity*
 - Crée log dans *ACCESS_LOGS*
6. Retourne succès/échec

Réponse succès :

```
{
  "status": "success",
  "message": "Action authorized",
  "user": "pierre",
  "locker_id": "WH-001",
  "action": "take",
  "item_id": 10,
  "quantity": 3
}
```

Réponse échec :

```
{
  "status": "denied",
  "message": "Permission denied",
  "reason": "User does not have can_take permission",
  "user": "pierre",
  "locker_id": "WH-001"
}
```



6.6.2.Armoires accessibles par l'utilisateur (Admin)

- GET /api/user/lockers : Liste les armoires accessibles par un utilisateur

Résultat :

```
{
  "user_id": "uuid-pierre",
  "username": "pierre",
  "lockers": [
    {
      "locker_id": "WH-001",
      "name": "Armoire A - Atelier",
      "location": "Bâtiment A",
      "locker_type": "warehouse",
      "permissions": {
        "can_view": true,
        "can_open": true,
        "can_take": true,
        "can_return": true,
        "can_manage": false
      }
    },
    {
      "locker_id": "OFF-001",
      "name": "Armoire B - Bureau",
      "location": "Bâtiment B",
      "locker_type": "office",
      "permissions": {
        "can_view": true,
        "can_open": false,
        "can_take": false,
        "can_return": false,
        "can_manage": false
      }
    }
  ]
}
```



6.7. Gestion des Permissions (Admin)

- GET /admin/permissions : Liste toutes les permissions
- GET /admin/permissions?locker_id=WH-001 // Permissions d'une armoire
- GET /admin/permissions?role_name=CODIR // Permissions d'un role
- GET /admin/permissions/{permission_id} : Détails d'une permission
- POST /admin/permissions : Crée une permission

```
{
  "locker_id": "WH-001",
  "role_name": "CODIR",
  "can_view": true,
  "can_open": true,
  "can_take": true,
  "can_return": true,
  "can_manage": false,
  "valid_from": "08:00",
  "valid_until": "18:00",
  "valid_days": "MON,TUE,WED,THU,FRI"
}
```
- PUT /admin/permissions/{permission_id} : Met à jour une permission
- DELETE /admin/permissions/{permission_id} : Supprime une permission



6.8. Audit & Logs

- GET /audit/logs : Liste les logs d'accès (avec filtres)
- GET /audit/logs?locker_id=WH-001 // Accès à une armoire
- GET /audit/logs?user_id=uuid-pierre // Actions d'un utilisateur
- GET /audit/logs?action=TAKE // Type d'action
- GET /audit/logs?start_date=2025-01-01&end_date=2025-01-31 // Période

Résultat :

```
{
  "logs": [
    {
      "log_id": 1001,
      "locker_id": "WH-001",
      "user_id": "uuid-pierre",
      "username": "pierre",
      "nfc_card_uid": "04:AB:CD:EF",
      "roles_snapshot": ["CODIR", "admin"],
      "action": "TAKE",
      "item_id": 10,
      "quantity": 3,
      "success": true,
      "timestamp": "2025-01-15T10:30:00Z"
    }
  ]
}
```

- GET /audit/logs/{log_id} : Détails d'un log



6.9. Code HTTP

Code	Meaning
200	OK : Succès
201	Created : Ressource créée
204	No Content : Suppression réussie
400	Bad Request : Données invalides
401	Unauthorized : Non authentifié
403	Forbidden : Authentifié mais pas autorisé
404	Not Found : Ressource introuvable
409	Conflict : Conflit (ex: stock insuffisant)
500	Internal Server Error : Erreur serveur